Estudo de caso

implementação e demonstração

4. ESTUDO DE CASO E VALIDAÇÃO EXPERIMENTAL

Para verificar a viabilidade da topologia, foi construída uma prova de conceito (PoC) que implementa o núcleo da PASE e o aplica à evolução de um produto real. O objetivo é demonstrar que as três camadas funcionam encadeadas, de ponta a ponta, sob governança humana e com rastreabilidade integral.

4.1 Implementação da Prova de Conceito

A PoC separa dois sistemas. O FinTrack é o produto-alvo: um painel financeiro pessoal (Next.js e FastAPI) com dashboard, transações, orçamentos, metas, relatórios e suporte. O PASE é o motor que o evolui: um núcleo em FastAPI sobre PostgreSQL com pgvector, organizado nas três camadas do Capítulo 3, e um painel de controle para o operador humano. Toda chamada a modelo de linguagem passa por um cliente único que registra tokens, custo e rastreio, e cada decisão relevante gera um registro em banco, da proposta bruta à decisão humana final. A validação apoia-se em execuções reais registradas no repositório, em oito roteiros de demonstração executáveis e em cerca de 450 testes automatizados. O Quadro 1 delimita o que a PoC implementa em relação à topologia proposta.

Quadro 1 – Componentes validados na PoC e escopo remanescente

Camada	Validado na PoC	Fora do escopo da PoC
Tarefas	Seis origens de demanda; normalização; Avaliador de Zonas; Memória Histórica, Coerência e Qualificação; bypass de engenharia	Prospecção Competitiva; recompensa por contribuição; alternância de modelos
Orquestração	Priorização com urgência; ciclo de vida por estados; gates humanos; reexecução e trava de custo; homologação; grafo de dependências; auditoria e reavaliação (por comando)	Pesos e limiares no DAG; sub-DAGs; replanejamento automático; agendamento
Execução	Planejamento e execução separados; isolamento por clone e branch; verificação automática; revisão independente; entrega por develop, release e principal	Contêiner efêmero por tarefa; ambiente de preview
UPA	Seis zonas autônomas; payload declarativo; validação em duas camadas; fallback nativo	Zonas conversacionais; Event Bus; fluxos declarativos; filtro de prompt injection

Camada	Validado na PoC	Fora do escopo da PoC
Tarefas	Seis origens de demanda; normalização; Avaliador de Zonas; Memória Histórica, Coerência e Qualificação; bypass de engenharia	Prospecção Competitiva; recompensa por contribuição; alternância de modelos
Orquestração	Priorização com urgência; ciclo de vida por estados; gates humanos; reexecução e trava de custo; homologação; grafo de dependências; auditoria e reavaliação (por comando)	Pesos e limiares no DAG; sub-DAGs; replanejamento automático; agendamento
Execução	Planejamento e execução separados; isolamento por clone e branch; verificação automática; revisão independente; entrega por develop, release e principal	Contêiner efêmero por tarefa; ambiente de preview
UPA	Seis zonas autônomas; payload declarativo; validação em duas camadas; fallback nativo	Zonas conversacionais; Event Bus; fluxos declarativos; filtro de prompt injection

Fonte: elaborado pelos autores (2026).

O painel de controle concentra a supervisão humana e torna o funcionamento do motor observável (Figuras 1 a 4).

Figura 1 – Descartadas: propostas barradas pelo funil por duplicidade ou incoerência, com o motivo de cada descarte

[Captura: Tarefas > Descartadas]

Fonte: elaborado pelos autores (2026).

Figura 2 – Dependências: grafo das tarefas, com nós coloridos por estado e arestas de dependência

[Captura: Dependências]

Fonte: elaborado pelos autores (2026).

Figura 3 – Sistema: fila de execução, memória vetorial, custo acumulado, guardrails e prompts versionados

[Captura: Sistema]

Fonte: elaborado pelos autores (2026).

Figura 4 – Contexto: documentos do produto versionados no banco e as etapas do ciclo que leem cada um

[Captura: Contexto]

Fonte: elaborado pelos autores (2026).

4.2 Fluxo 1: da Demanda ao Código em Produção

O primeiro fluxo acompanha uma funcionalidade real, o modo escuro do FinTrack (tarefa nº 6 do PASE), da demanda em linguagem natural até a publicação na versão 1.0.0 do produto.

Figura 5 – FinTrack antes da solicitação: interface apenas em tema claro, sem controle de tema

[Captura: FinTrack em /dashboard no commit 09dff0d]

Fonte: elaborado pelos autores (2026).

A demanda entra no painel em texto livre e atravessa o Motor de Filtragem: normalização, Memória Histórica (busca vetorial contra itens implementados, em andamento e descartados), Coerência Sistêmica frente aos documentos de contexto e Qualificação por impacto, complexidade, confiança e risco. O resultado é uma tarefa candidata priorizada, que aguarda a decisão do gestor (Figura 6).

Figura 6 – Painel do PASE recebendo a solicitação: demanda em linguagem natural e tarefa candidata qualificada, aguardando aprovação ou recusa

[Captura: Tarefas > Nova demanda e aba Candidatas]

Fonte: elaborado pelos autores (2026).

Aprovada a candidata, a cadeia segue de forma autônoma até o próximo ponto de decisão humana. Na Fase de Planejamento, o Executor mapeia os arquivos afetados, estima o custo e define critérios de aceite antes de escrever código (Figura 7). Na Fase de Execução, trabalha em clone isolado e branch exclusiva, e a entrega só chega ao humano depois de passar pela verificação automática: tipos, testes, build de produção, preservação dos testes existentes, teste novo para comportamento novo e cobertura não decrescente. Tentativas e custo têm tetos impostos por código (Figura 8).

Figura 7 – Documento de Planejamento Técnico: arquivos previstos, critérios de aceite (C1 a C5) e trilho do ciclo de vida da tarefa

[Captura: tarefa nº 6, aba Planejamento]

Fonte: elaborado pelos autores (2026).

Figura 8 – Execução registrada: diff, log das verificações automáticas e transcrição integral da interação com o modelo

[Captura: tarefa nº 6, aba Execuções]

Fonte: elaborado pelos autores (2026).

A entrega alterou sete arquivos do frontend: definiu a paleta escura, adicionou o botão de alternância e persistiu a preferência por persona, com um teste para cada um dos cinco critérios de aceite. Um agente revisor separado conferiu cada critério citando o código e o teste correspondentes, e o sistema verificou por código que as citações existem. Só então o gestor decidiu (Figura 9).

Figura 9 – Homologação: parecer da revisão independente por critério de aceite e decisão do gestor (aprovar, ajustar ou recusar)

[Captura: tarefa nº 6, aba Homologação]

Fonte: elaborado pelos autores (2026).

Aprovar não equivale a publicar. A tarefa é integrada na develop, aprovada funcionalmente, incluída na release/1.0.0, validada com as suítes completas e só então publicada na branch principal, mediante nova aprovação humana (Figura 10). O histórico do repositório preserva essa cadeia com autoria do agente (commits 7c810b6, 0d6050a e 0ff8483), e a funcionalidade passa a valer no produto em execução (Figura 11).

Figura 10 – Tela Entrega: tarefa na develop, release 1.0.0 validada e publicação na branch principal sob aprovação humana

[Captura: Entrega, com a release 1.0.0]

Fonte: elaborado pelos autores (2026).

Figura 11 – FinTrack após a entrega: botão "Usar tema escuro" e paleta escura aplicada a todas as telas

[Captura: FinTrack em /dashboard no estado atual, tema escuro, mesmo enquadramento da Figura 5]

Fonte: elaborado pelos autores (2026).

4.3 Fluxo 2: Personalização em Tempo de Execução

O segundo fluxo valida o UPA. Sem tocar o repositório e sem gate humano, ele personaliza as Zonas Dinâmicas do FinTrack (seis, uma por tela) preenchendo propriedades de um catálogo fechado de três widgets. A telemetria de navegação aciona a regra da zona, o UPA gera um payload declarativo e o servidor o valida contra o contrato antes de persisti-lo (Figura 12).

Figura 12 – Simulador do painel: o uso intenso de relatórios pela persona Ana aciona a zona do dashboard e o UPA gera um payload declarativo

[Captura: Zonas > Simulador e payload ativo com o JSON expandido]

Fonte: elaborado pelos autores (2026).

No cliente, uma segunda validação, a decisiva, confronta o payload com o contrato (Hard Constraints) antes de renderizá-lo. A personalização aparece apenas para quem a motivou (Figura 13).

Figura 13 – Hiper-personalização em execução: Ana vê o widget gerado pelo UPA, com o selo "personalizado", enquanto Clara mantém o componente nativo

[Captura: FinTrack em /dashboard como Ana e como Clara, lado a lado]

Fonte: elaborado pelos autores (2026).

A defesa foi testada no pior caso: payloads maliciosos gravados diretamente no banco, contornando o servidor (rótulo de 500 caracteres, link externo e widget fora do catálogo). O cliente recusou os três, manteve o componente nativo e registrou cada violação (Figura 14). Em nenhum momento a IA gera código executável.

Figura 14 – Defesa em profundidade: payload com link externo injetado sem validação do servidor, recusado pelo cliente e registrado como violação

[Captura: Zonas > Injetor e lista de Violações]

Fonte: elaborado pelos autores (2026).

4.4 Discussão

Os dois fluxos demonstram que o núcleo da topologia funciona: uma demanda em linguagem natural percorreu as três camadas e chegou à produção com decisão humana em cada ponto crítico, e a personalização em tempo de execução operou sem expor o núcleo a código arbitrário. O desenvolvimento também confirmou na prática o princípio de contenção da generatividade. O próprio modo escuro exigiu quatro tentativas, e as falhas anteriores, uma entrega que quebrou o aplicativo e outra que passou nos testes sem definir nenhuma cor, só deixaram de se repetir quando as garantias saíram das instruções ao modelo e passaram a ser verificadas por código (ABOU ALI; DORNAIKA, 2025). As limitações desta validação são discutidas na Seção 5.2.

Trabalhos relacionados

1 paragrafo por "concorrente"

TRABALHOS RELACIONADOS

Esta seção apresenta os trabalhos que mais se aproximam da proposta, selecionados por cobrirem, cada um, uma das dimensões centrais da Arquitetura PASE: a supervisão humana no fluxo de desenvolvimento com agentes, a automação do ciclo de vida por sistemas multiagente, a originação de demandas a partir de fontes não estruturadas e a personalização de interfaces em tempo de execução. Para cada trabalho, descrevem-se sua proposta, os pontos de convergência com a PASE e as diferenças que delimitam a contribuição deste estudo.

O trabalho mais próximo da PASE em termos de supervisão humana é o HULA (*Human-in-the-Loop LLM-based Agents*), proposto por Takerngsaksiri et al. (2025) e implantado em produção no Jira da Atlassian. O *framework* organiza-se em três agentes que cooperam para resolver uma *issue*: o *AI Planner Agent*, que localiza os arquivos afetados e gera um plano de codificação; o *AI Coding Agent*, que implementa o plano; e o *Human Agent*, o engenheiro que revisa, refina e aprova tanto o plano quanto o código antes que um *pull request* seja aberto. Essa sequência corresponde diretamente ao trecho que vai da Camada de Orquestração à Camada de Execução da PASE, em especial à exigência de um Documento de Planejamento Técnico prévio à escrita de código e à etapa de homologação humana com autoridade final. O HULA, contudo, inicia seu fluxo a partir de uma *issue* já formulada e priorizada por humanos, não trata a originação nem a filtragem de demandas, não modela interdependências entre tarefas, não possui memória das rejeições e não atua sobre o software em tempo de execução. A PASE pode ser entendida como uma generalização do princípio do HULA, deslocando a supervisão humana de pontos isolados de revisão para um plano de controle que acompanha a tarefa desde sua captação até a personalização em produção.

Na dimensão da amplitude do ciclo de vida coberto, destaca-se o MetaGPT (HONG et al., 2024), o *framework* multiagente que mais se aproxima da PASE. Nele, agentes assumem papéis de uma equipe de software (gerente de produto, arquiteto, gerente de projeto, engenheiro e analista de qualidade) e seguem Procedimentos Operacionais Padronizados (SOPs), de modo que um requisito de uma linha é progressivamente transformado em documento de requisitos, projeto de sistema, lista de tarefas, código e testes, com artefatos estruturados trocados entre os papéis em linha de montagem para reduzir a propagação de alucinações. A decomposição de uma demanda em etapas com artefatos intermediários verificáveis e a separação entre quem planeja e quem executa são princípios compartilhados pela PASE, cuja Camada de Orquestração também produz documentos de planejamento antes da implementação. A diferença central está no papel do ser humano e na origem das demandas: o MetaGPT busca simular uma organização inteira de software com agentes e concentra a intervenção humana no requisito inicial, enquanto a PASE mantém o gestor humano como responsável pela priorização e pela homologação e trata a estratégia de coordenação entre agentes como detalhe interno do Executor, em consonância com a evidência de que a eficácia de sistemas multiagente depende da

adequação entre a estrutura de coordenação e a tarefa, e não do número de papéis envolvidos (KIM et al., 2026). Além disso, o MetaGPT não prevê isolamento obrigatório de execução, controle de custo, auditoria recorrente da base de código nem adaptação em tempo de execução.

No que diz respeito à originação e qualificação de demandas, a proposta que mais se aproxima da Camada de Tarefas da PASE é o LLM-Cure, de Assi, Hassan e Zou (2025). A abordagem utiliza modelos de linguagem para extrair funcionalidades mencionadas em avaliações de usuários em lojas de aplicativos e, diante de uma reclamação sobre determinada funcionalidade, recupera avaliações bem pontuadas de aplicativos concorrentes para sintetizar sugestões concretas de melhoria. A avaliação em mais de um milhão de avaliações de setenta aplicativos Android indicou que a maior parte das sugestões geradas foi posteriormente implementada pelos próprios desenvolvedores, o que evidencia a viabilidade de converter *feedback* informal e dados de concorrentes em propostas acionáveis. Esse desenho corresponde, na PASE, à captação passiva do Fluxo de Agentes Externos e à Prospecção Competitiva do Fluxo da Inteligência Artificial. O LLM-Cure, entretanto, restringe-se a uma única fonte de insumos e encerra seu ciclo na recomendação ao desenvolvedor, não contemplando a normalização de entradas heterogêneas, a deduplicação contra o histórico de funcionalidades e propostas descartadas, a verificação de coerência com as regras de negócio nem a atribuição de metadados de impacto, complexidade e confiança. A PASE estende essa linha ao tratar a mineração de *feedback* e o *benchmarking* como apenas duas das fontes de um *pipeline* unificado de filtragem progressiva, cujo produto é um *backlog* qualificado e pronto para priorização sob supervisão humana, e não uma lista de sugestões isoladas.

A personalização em tempo de execução encontra seu precedente acadêmico mais direto em Cao, Jiang e Xia (2025), apresentado na CHI 2025. Os autores propõem o conceito de interfaces generativas e maleáveis: em vez de telas estáticas definidas em tempo de projeto, a interface é derivada de um modelo de dados orientado à tarefa, gerado por um LLM a partir do *prompt* do usuário e composto pelas entidades, relações e propriedades relevantes à atividade em curso. Esse modelo pode evoluir à medida que o usuário refina sua intenção, e cada alteração se propaga para a interface e para os dados subjacentes. O protótipo Jelly e o estudo com oito participantes indicaram que os usuários valorizaram a combinação entre elementos estáticos previsíveis e elementos gerados sob demanda, resultado que converge com a decisão da PASE de preservar o comportamento nativo padrão e abrir à personalização apenas áreas delimitadas. A ideia de separar uma representação estruturada do estado da tarefa, que o modelo gera, da sua renderização, que o sistema controla, é também o fundamento dos *payloads* declarativos das Zonas Dinâmicas, inclusive quando estes descrevem máquinas de estado para fluxos de múltiplas etapas. O trabalho de Cao, Jiang e Xia, porém, situa-se na perspectiva da interação humano-computador e parte sempre de uma intenção explícita do usuário, correspondendo apenas ao que a PASE denomina Zonas Sob Demanda; não há equivalente às Zonas Autônomas, acionadas pelo UPA a partir de telemetria de frustração, nem ao Avaliador de Zonas Dinâmicas, que decide, com base no padrão das demandas recebidas, quais módulos devem se tornar parametrizáveis e quais devem receber uma funcionalidade nativa. Tampouco são tratados os mecanismos de governança próprios de software comercial multiusuário: contratos rígidos por zona com aborto da renderização e retorno ao padrão nativo, mediação do acesso a dados por um SDK tipado sob menor privilégio,

filtragem prévia de injeção de *prompt* e integração entre componentes dinâmicos e estáticos por meio de um barramento local de eventos. A PASE, portanto, incorpora a maleabilidade proposta pelos autores, mas a subordina a uma camada de governança que define onde, quando e sob quais limites a geração pode ocorrer.

Em conjunto, os trabalhos analisados mostram que cada dimensão da PASE possui precedentes, mas nenhum deles a cobre por completo. O HULA insere o humano na revisão de plano e código, mas não na priorização nem na memória das decisões rejeitadas; o MetaGPT decompõe o ciclo de vida em papéis e artefatos, mas sem isolamento de execução, controle de custo ou auditoria da base de código; o LLM-Cure converte *feedback* disperso em sugestões, mas sem normalização multifonte, deduplicação contra o histórico ou qualificação por metadados; e o trabalho de Cao, Jiang e Xia torna interfaces maleáveis por intenção do usuário, mas sem gatilho por telemetria, sem critério para decidir quais áreas devem ser parametrizáveis e sem contratos que limitem a geração. A contribuição da PASE, portanto, não se resume a integrar essas frentes: dentro de cada camada ela introduz mecanismos que os trabalhos relacionados não contemplam e, entre as camadas, subordina todos eles a um mesmo regime de autonomia supervisionada, rastreabilidade e contenção da generatividade.

REFERÊNCIAS

ASSI, M.; HASSAN, S.; ZOU, Y. LLM-Cure: LLM-based competitor user review analysis for feature enhancement. *ACM Transactions on Software Engineering and Methodology*, New York, 2025. DOI: 10.1145/3744644.

CAO, Y.; JIANG, P.; XIA, H. Generative and malleable user interfaces with generative and evolving task-driven data model. In: CHI CONFERENCE ON HUMAN FACTORS IN COMPUTING SYSTEMS (CHI ‘25), 2025, Yokohama. *Proceedings* […]. New York: ACM, 2025. art. 686, p. 1-20. DOI: 10.1145/3706598.3713285.

HONG, S. et al. **MetaGPT**: meta programming for a multi-agent collaborative framework. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS, 12., 2024, Viena. *Proceedings* […]. [S. l.]: ICLR, 2024. Disponível em: https://openreview.net/forum?id=VtmBAGCN7o. Acesso em: 17 out. 2026.

TAKERNGSAKSIRI, W. et al. Human-in-the-loop software development agents. In: IEEE/ACM INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING: SOFTWARE ENGINEERING IN PRACTICE (ICSE-SEIP), 47., 2025, Ottawa. *Proceedings* […]. Piscataway: IEEE, 2025. DOI: 10.1109/ICSE-SEIP66354.2025.00036. Disponível em: https://arxiv.org/abs/2411.12924. Acesso em: 17 out. 2026.